

NYCU-ECE DCS-2026

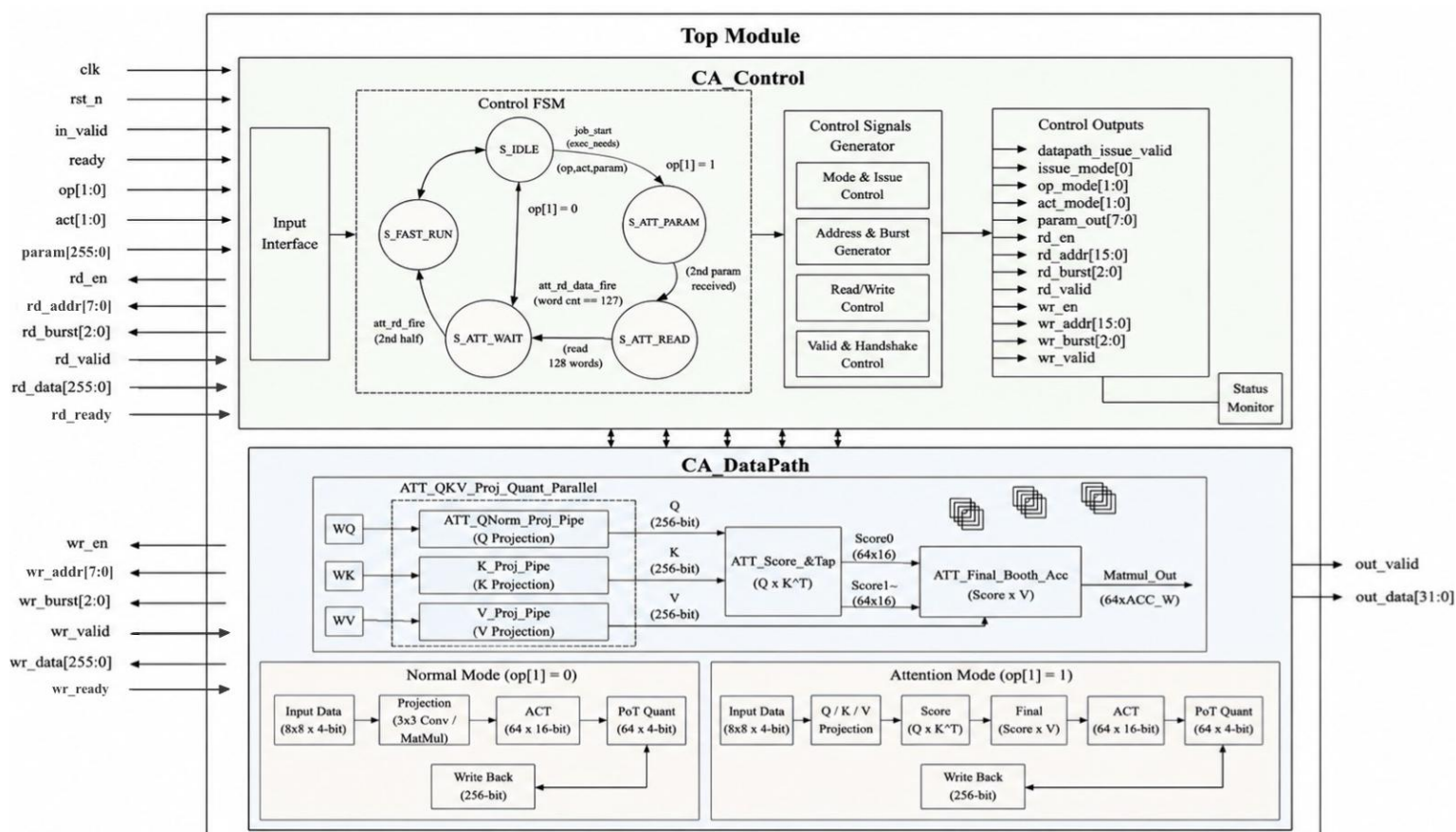
Final Project Report

Name: 楊庭瑞

Student ID: 113511266

Server Account: dcs151

Architecture (2 points)



Design Methodology (8 points)

1. Data Flow (FSM, or how your design process data):

本次專題將主要架構拆分成 **CA_Control** 與 **CA_DataPath**。**CA_Control** 負責整體控制流程，包含 FSM 狀態轉移、RAM read/write command 產生、參數暫存，以及 datapath issue timing 的控制。**CA_DataPath** 則負責實際資料運算，包含 matrix/convolution projection、attention computation、activation、PoT quantization，以及最後的資料輸出與寫回。FSM 主要包含五個狀態：`S_IDLE`、`S_FAST_RUN`、`S_ATT_PARAM`、`S_ATT_READ`、`S_ATT_WAIT`。當系統位於 `S_IDLE`，且 `mem_set` 與 `in_valid` 被拉起時，控制器會接收新的 operation set，

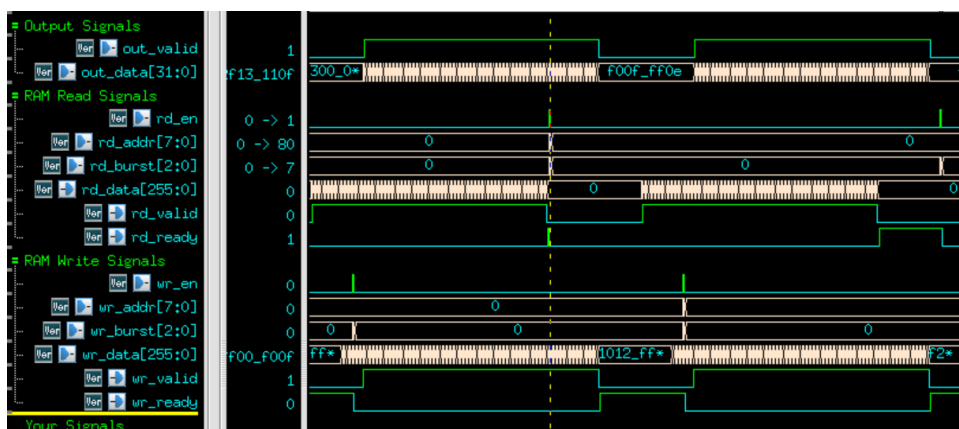
並將 op、act 與第一筆 param 暫存起來。接著 FSM 會根據 op[1] 決定資料要走一般運算流程，或是 attention 運算流程。

- 當 op[1] = 0 時 (FFN and CONV)，FSM 會進入 S_FAST_RUN，控制器會對 RAM 發出兩次 128-word burst read command，分別讀取 RAM 的前半部與後半部資料，共處理 256 筆 RAM word。每當 RAM 回傳有效資料 rd_valid 時，控制器便會送出 IM_NORM issue mode，讓 datapath 接收該筆資料並進行一般 projection 或 convolution 運算。運算結果接著會通過 activation module，再經過 PoT quantization 量化回 signed 4-bit 格式，最後重新打包成 256-bit word 寫回 RAM，並透過 out_data 輸出最低 32-bit 結果。
- 當 op[1] = 1 (SHA and MHA) 時，由於 Attention 是輸入額外的權重矩陣，因此 FSM 會先進入 S_ATT_PARAM，依序接收並暫存 WK 與 WV。其中第一筆 param 會作為 WQ，後續兩筆 param 分別作為 WK 與 WV。完成參數接收後，FSM 會進入 S_ATT_READ，開始從 RAM 讀取 input data。DataPath 會先進行 Q、K、V projection，接著由 ATT_Score_8Tap 計算 Q 與 K 的 score，再由 ATT_Final_Booth_Acc 將 score 與 V 相乘並累加，產生 attention output。最後同樣經過 activation 與 PoT quantization，將結果寫回 RAM 並輸出。

2. Execution Cycle Optimization (how you minimize execution cycle):

在運算流程，不論何模式，FSM 皆使用兩次 128-word burst read 來完成 256 筆資料的讀取，直接把 burst 拉到最大，這樣可以大量降低 latency，使資料能夠更連續地送入 Datapath。當 pipeline 被填滿後，Datapath 可以連續處理多筆資料，因此雖然第一筆輸出需要經過完整 pipeline latency，但後續資料可以以較高 throughput 連續輸出 1cycle/1word。

在 Attention 運算流程中，FSM 將 256 筆資料分成兩個 128-word half 來處理。第一半資料完成 read 與 computation 後，透過 S_ATT_WAIT 協調 write command，再接續處理第二半資料。這樣的流程可以避免 read、compute 與 writeback 之間產生過多 idle cycle。且由於 rd_en 發出後會延遲 50 cycle，我也設計了第二次的接收 (129~256word) 可以在前面還沒算完的時候就先發出，這樣等算完沒多久之後，新的 word 就進來了，減少資源等待時間。



在上筆輸出還沒結束先拉高 rd_en

3. Cycle Time Optimization (how you minimize cycle time):

為了降低 cycle time，我把所有切分成多個 pipeline stage。這樣可以減少單一 clock cycle 內需要完成的 critical path，使電路能夠在較短的 clock period 下運作。

- 在乘法階段中，乘法與加總運算並不是全部放在同一個 cycle 中完成，而是透過多級 pipeline 分開處理。輸入資料與權重會先被暫存，接著進行乘法運算，再將 partial sum 分階段累加，最後才進行量化。這樣可以避免 multiplier 與大型 adder tree 形成過長的 critical path。
- ACT_5Stage_Parallel 採用了五級 pipeline 架構。對於 RAT、CAT 與 BAT 這類需要計算 threshold 的 activation function，若直接在一個 cycle 內完成 row sum、column sum 或 block sum，會造成很長的加法器路徑。因此先進行 pair sum，再進行 partial sum，接著產生 threshold，最後才套用 activation function。透過這樣的分級處理，可以有效降低 activation 階段的 timing burden。
- PoT 的目標是根據資料的最大有效位元決定 shift amount，若直接使用完整 comparator tree 尋找 64 個元素中的最大值，會造成較大的面積與較長的 delay。最後使用 OR-reduction 的方式找出有效 MSB 位置，再決定 shift amount，最後進行 arithmetic shift 與 signed 4-bit clamp。這種方式比完整大小比較器更適合 timing optimization。
- 在 attention final stage 中，使用 Booth recoding 來簡化 score 與 4-bit V value 的乘法。Booth partial product generation、product combination、partial accumulation 與 final accumulation 被分成不同 pipeline stage，因此可以降低單級乘加路徑的延遲。

4. Area Optimization (how you reduce your design area):

為了提高 throughput 使用了高度平行化架構，因此面積優化非常有限

- RAM 中的資料採用 signed 4-bit packed format 儲存。一個 256-bit RAM word 內含 64 個 signed 4-bit element。這樣可以大幅降低記憶體資料寬度與儲存成本。只有在 datapath 內部進行乘加累積時，才使用較寬的 bit width 以保留計算精度。換句話說，我將高精度資料限制在必要的運算階段，而不是整體都使用寬 bit-width，藉此降低面積。
- DataPath 中的 Q projection path 同時被 normal operation 與 attention operation 重複使用。在 normal mode 中，這條 projection path 用來產生 FFN 或 convolution 的運算結果；在 attention mode 中，則用來產生 Q matrix。透過共用乘法器，可以避免為一般運算與 attention Q computation 各自建立一套獨立乘加架構，不過為了保證可以連續 128 筆 output 輸出，除了跟 FFN Conv 共用乘法器，其他 ATT 的計算都是獨立的乘加器。
- PoT quantization 使用 OR-reduction 來判斷 shift amount，而不是使用完整 maximum comparator tree。由於 PoT 量化只需要知道最高有效位元的位置，不一定需要精確比較所有 64 個元素的大小，因此 OR-reduction 可以用較少的硬體達到所需功能。

Discussion (3 points)

Difficulties encountered and how you solved them.

這次最難的我覺得是在 performance 的 trade-off，我試了很多種版本，一開始一次收一筆資料，到後來 ATT 一次收 4 筆，一般路徑收 128 筆，發現壓 latency 應該是最划算的，就果斷往 ATT 也一次收 128 筆去進行，用大量的 pipeline parallel 的概念，因此也導致面積變蠻大的，不知道怎麼壓進去，乘法器也沒辦法共用的很多，最後也只能放著很大的面積 1000 多萬，每次跑合成都要跑很久，剛寫出來的時候甚至還合成超過 2 小時，後來優化後合成大概 40 分鐘，不過雖然把面積放掉 performance 會比較好，原本想要全部階段都共用乘法器，結果最後發現 ATT 乘法器全部都不要共用 performance 竟然比較好（可能是我寫錯 XD）。

Reflection (2 points)

在修這堂課之前，就有聽說過這是一堂很硬的課，自己從學期初就在想要要不要退選（二下的必修好多，這學期不小心修了 26 學分 Q），但還是很開心自己可以堅持到最後 XD，也感謝所有認真出題的助教，這次的 Final Project 我花了蠻多時間，想了很多架構，也讓我學習到了很多切 pipeline 技巧，以及很多設計 trade-off 的考量，追求什麼變小就要付出什麼代價真的很難，由於程式很龐大，大概 2500 行，因此我覺得 coding style 特別重要，否則寫著寫著就不知道自己在寫什麼，慢慢跟自己的程式碼脫節，不過也是讓我見識到 AI 的強大，自己寫根本就寫不贏 XD。

小彩蛋（看助教 Pass Pattern 太可愛也是自己玩上了~）



AI-use Log

If AI was used, please include the following details:

- Share the **link** to the AI chat (do not just copy and paste the webpage URL).
https://github.com/xiaotin22/NYCU_DCS_2026/blob/main/DCS_FINAL/AI_uselog.md
- Key Prompts: 3–5 primary prompts (paste the **screenshots**).
 - 「分析一下目前 FINAL CA.sv 的程式架構」
 - 「幫我時序分析一下目前 resource 的使用情況以及 latency cycle 總數為啥是那樣」
 - 「幫我根據 critical path 切 pipeline」
 - 「MHA head0 partial 怎麼借 q_mem / k_mem 才不會跟下一組 QKV 衝到」
 - 「Matrix_Max 改用 OR-reduction 代替 max-tree 的正確性怎麼證明」